



S.I.G.P.D.

Administración de Sistemas Operativos

CEITAROX

Rol	Apellido	Nombre	C.I	Email
Coordinador	Rodríguez	Piero	5.592.003-7	rpierina286@gmail.com
Sub-Coordinador	Gracés	Tatiana	5.713.652-9	tatianatareasutu@gmail.com
Integrante 1	Cardozo	Celeste	5.716.176-6	celestecardozo1803@gmail.com
Integrante 2	Vázquez	Ignacio	5.479.448-7	nachosfera14@gmail.com

Docente: Martínez, Santiago

**Fecha de
culminación
15/09/2025**

SEGUNDA ENTREGA



Índice

1. Desarrollo del proyecto.....	2
2. Justificación de la elección del sistema operativo.....	3
3. Manual de instalación de Fedora.....	4
3.1. Instalar el sistema operativo Fedora en PC.....	4
3.2. Instalar Fedora Server en VMware Workstation Pro.....	6
4. Creación de script de suba y baja de usuarios y grupos.....	12
4.1 Menú principal.....	12
4.2 Menú de gestión de usuarios y grupos.....	14
5. Instalación de paquetes.....	19
6. Revisión del firewall, rich rules e iptables.....	22
7. Configuración del protocolo SSH.....	23
8. Implementación de clave pública-privada.....	26
8.1 Conectividad de la máquina física con el servidor.....	27
8.2 Testeo de conexión.....	28
8.3 Traslado y adaptación de la clave pública a el servidor.....	29
8.4 Testeo final del sistema de autenticación.....	30
9. Continuación del script (creación de las funciones de firewalld y respaldo).....	31
9.1 Firewall.....	31
9.2 Respaldo.....	31
10. Instalación de Docker Engine.....	32
10.1 Instalación de las imágenes de docker.....	34
11. Creación de los contenedores.....	36
11.1 Contenedor con imagen de PHP 8.2 y Apache.....	36
11.2 Contenedor con imagen MySQL.....	39
12. Bibliografía.....	41

CEITAROX

I.S.B.O.

3MJ



1.0 Desarrollo del proyecto

En esta carpeta se expondrá nuestro proyecto de egreso, con el nombre de CEITAROX presentamos un sistema de seguimiento y gestión para el juego de mesa de tipo "draft" de nombre Draftosaurus de manera online, el cual hemos bautizado como FlowerDraft, donde ganará el jugador con la mejor florería.

En esta carpeta presentaremos el procedimiento de creación de un servidor desde una máquina virtual que permitirá el acceso remoto de los usuarios finales a la base de datos de la aplicación. Se detallará el transcurso de la instalación de su sistema operativo, sus paquetes, la configuración y habilitación del protocolo SSH, la implementación de scripts para el manejo automatizado del servidor por parte de los integrantes, la aplicación de la clave pública-privada, la instalación del servicio Docker y la creación de los primeros contenedores.

CEITAROX

I.S.B.O.

3MJ



2.0 Justificación de la elección del sistema operativo

El grupo de proyecto eligió la línea de Linux Fedora, específicamente la versión para servidores número 42 (Fedora Server 42), porque:

- Nuestro profesor de Adm. de Sistemas Operativos nos enseñó cómo utilizar Fedora, por lo tanto estamos más familiarizados con este sistema operativo.
- Nuestro enfoque se centra en la creación, gestión, configuración y comunicación de un servidor remoto, por ende no precisamos de una línea genérica de Fedora.
- Ofrece el uso de diversas tecnologías adaptadas en forma de paquetes, como por Ej. Cockpit, que contribuyen a una gestión y seguridad robusta.
- Pertenece a la familia RedHat, lo que nos permite trabajar con menos complicaciones en nuestras instrucciones con el servidor y los usuarios involucrados.
- Fedora usa un software moderno sin comprometer la estabilidad.
- Fedora tiene una interfaz intuitiva y fácil de usar.
- Tiene buenas herramientas, soporte para muchos lenguajes y contenedores integrados.
- Es un sistema operativo seguro y se actualiza frecuentemente.
- Con el comando DNF install puedes tener lenguajes como Python, Java, etc.



3.0 Manual de instalación de Fedora

3.1. Instalar el sistema operativo Fedora en PC

Requisitos:

- Una PC o laptop con al menos 2gb de RAM.
- USB de al menos 4GB..
- Acceso a internet (opcional, pero recomendable).

Paso 1: Descargar Fedora

1. Ve al sitio oficial <https://getfedora.org> . Haga click en “download now” bajo la opción “workstation”.

Paso 2: Crear USB booteable.

Usa uno de estos programas:

- Rufus (Windows)
- Balena Etcher (Windows/macOS/Linux)
- Fedora media writer (oficial)

Pasos (con Balena Etcher):

1. Abra Balena Etcher
2. Seleccione la ISO de Fedora que descargaste
3. Seleccione su USB
4. Realice click en “flash”

Paso 3: Instalar desde el USB

5. Inserte el USB en la computadora donde vas a instalarlo.
6. Reinicie y entre al menú de arranque (presionar las teclas F12, F2, esc, o DEL según tu pc)
7. Seleccione el USB

Paso 4: Cuando aparezca el menú de Fedora, seleccione: “Install to hard drive”

CEITAROX I.S.B.O. 3MJ



Paso 5: Elegir el idioma e iniciar instalación

8. Seleccione su idioma.
9. Haga click en “continue”

Paso 6: Configurar instalación

Debe revisar:

- *Destino de instalación (disco):* elija el disco donde piensa instalar Fedora.
- *Zona horaria:* seleccione su ubicación en el mapa.

Paso 7: Instalar Fedora

Haga click en “begin installation” y espere unos minutos a que termine.

Paso 8: Reiniciar el dispositivo

Cuando termine la instalación, haga clic en “finish installation” y reinicie el equipo. Retira el USB cuando te lo indique.

Paso 9: Configurar usuario

Al iniciar por primera vez cree su usuario y escoja una contraseña.

3.2. Instalar Fedora Server en VMware Workstation Pro

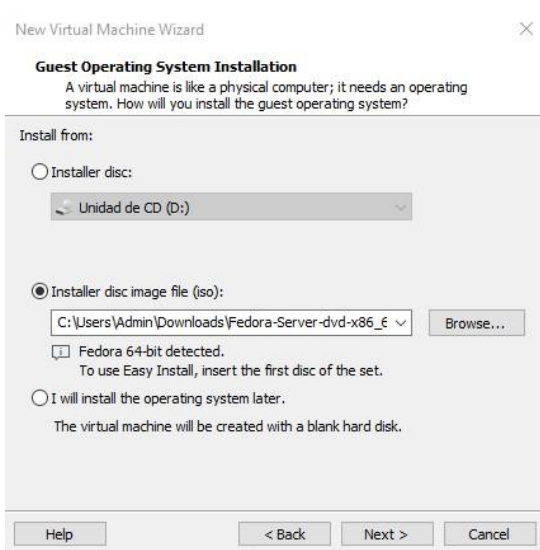
Debido a que algunos integrantes poseen otros sistemas operativos y desean conservarlos en sus dispositivos, hemos hecho este manual alternativo para poder instalar Fedora en la máquina virtual.

Antes de la instalación:

- Descargar el archivo .ISO de Fedora Server 42 desde la página oficial. Instalamos la versión Server y no la versión Workbench ya que no resulta necesario tener la interfaz gráfica, además Fedora Server pesa menos en el disco duro.
- Ya tener instalado VMware Workstation Pro



Luego de haber instalado el VMware y haber descargado el archivo .ISO, presionaremos el botón para crear una nueva máquina virtual, y cambiaremos el modo de creación a “Típico”.



El próximo paso es definir el tipo de disco duro que queremos en la máquina virtual, nosotros utilizaremos una imagen lógica, para ello hay que especificarlo con la ubicación donde se encuentra el .ISO que hemos descargado.

CEITAROX

I.S.B.O.

3MJ



15/09/25

New Virtual Machine Wizard

Name the Virtual Machine
What name would you like to use for this virtual machine?

Virtual machine name:

Location:

The default location can be changed at Edit > Preferences.

< Back Next > Cancel

En la siguiente pantalla le indicaremos el nombre de nuestra máquina virtual y la ubicación de dónde se ejecutará.

New Virtual Machine Wizard

Specify Disk Capacity
How large do you want this disk to be?

The virtual machine's hard disk is stored as one or more files on the host computer's physical disk. These file(s) start small and become larger as you add applications, files, and data to your virtual machine.

Maximum disk size (GB):

Recommended size for Fedora: 20 GB

☐ Store virtual disk as a single file

☒ Split virtual disk into multiple files

Splitting the disk makes it easier to move the virtual machine to another computer but may reduce performance with very large disks.

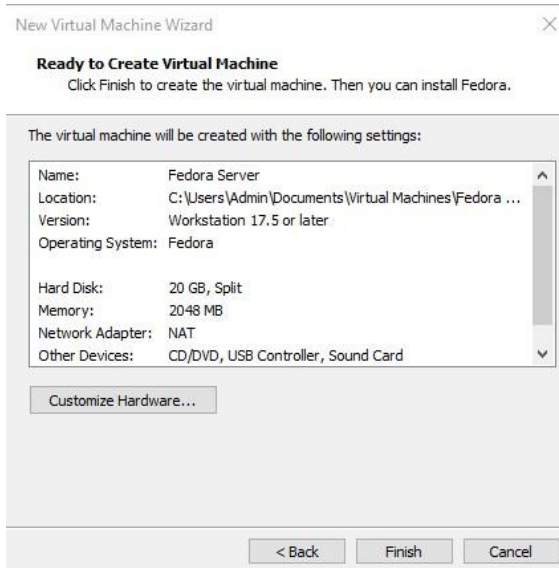
Help < Back Next > Cancel

En la pestaña de disco duro nosotros estableceremos 20GB de almacenamiento..

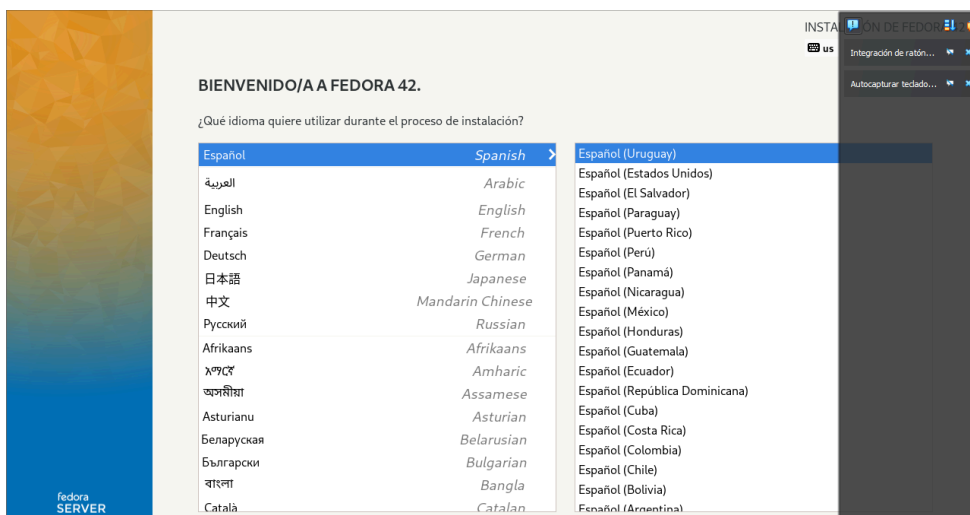
CEITAROX

I.S.B.O.

3MJ



Por último se muestra en forma resumida las configuraciones de hardware que hemos realizado en los pasos anteriores, si se desea cambiar cierta propiedad debe presionar el botón “Customize Hardware”.



Una vez iniciada, nos recibirá una interfaz gráfica, donde seleccionaremos como idioma el Español, y continuamos a la siguiente pantalla.

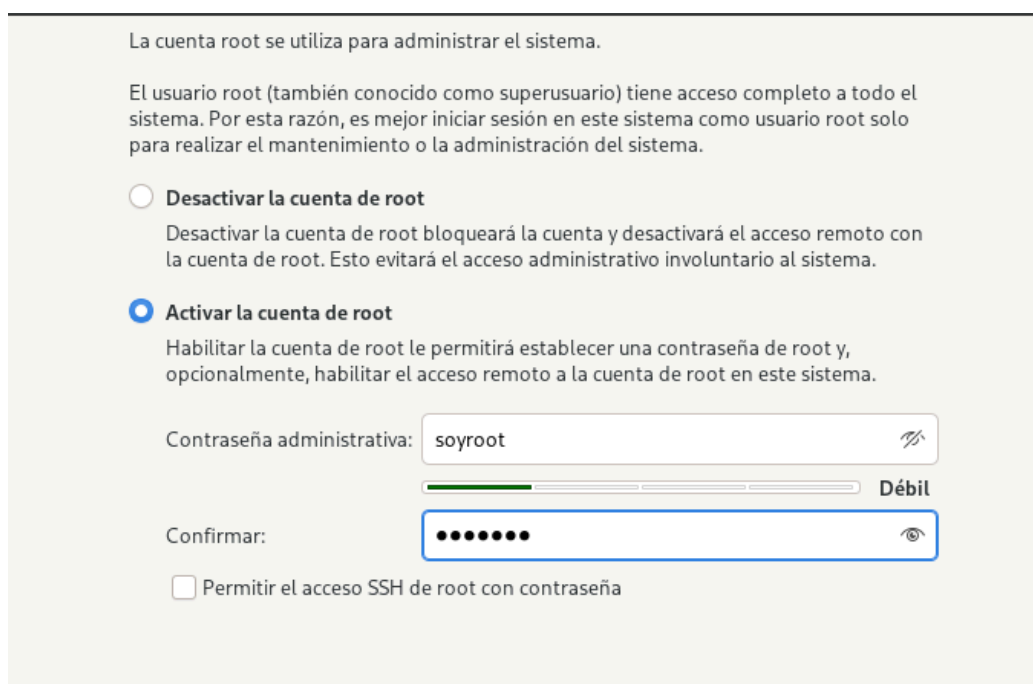
CEITAROX

I.S.B.O.

3MJ



Nos enfocaremos en las opciones de ajustes de usuario



CEITAROX

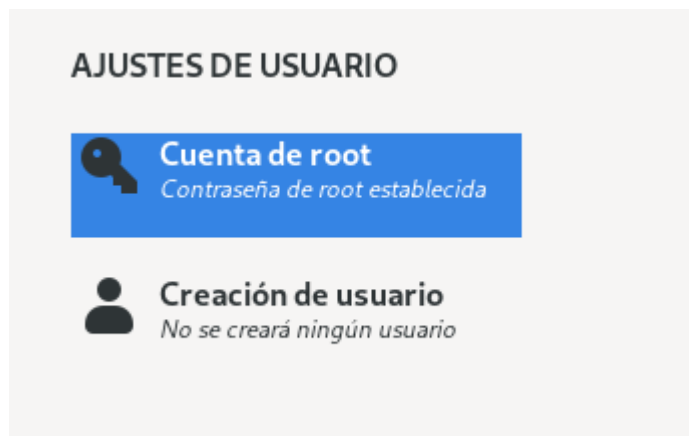
I.S.B.O.

3MJ



Crearemos una cuenta root para administrar, seleccionaremos la opción de “Activar la cuenta de root”, luego crearemos una contraseña; en la imagen se muestra un ejemplo de contraseña, sin embargo se recomienda crear una contraseña que no sea vista como débil por Fedora.

Una vez terminado presionamos el botón “hecho” para volver a la pantalla anterior y nos dirigiremos a “Creación de usuario”



Aquí mostramos un ejemplo de cuenta de usuario, donde activaremos la opción para que el usuario esté dentro del grupo wheel, esto lo haremos para evitar administrar y movernos por archivos utilizando el usuario root constantemente. Una vez terminado de crear el usuario con su respectiva contraseña, daremos de nuevo click al botón de “hecho”.



Selección de dispositivo
Seleccione los dispositivos en que le gustaría instalar. Se mantendrán sin tocar hasta que pulse el botón «Comenzar instalación» del menú principal.

Discos estándares locales

20 GiB

 **ATA VBOX HARDDISK**
sda / 20 GiB libre

Los discos que se dejen aquí sin seleccionar no se tocarán.

Discos especializados y de red

 **Añadir un disco...**

Los discos que se dejen aquí sin seleccionar no se tocarán.

Configuración de almacenamiento

☒ Automática ☐ Personalizada ☐ Personalizada avanzada (Blivet-GUI)

☐ Libere espacio eliminando o redimensionando particiones existentes

Cifrado

☐ Cifrar mis datos. *Usted fijará una frase de paso después.*

Extra: En destino de instalación hemos dejado las opciones por default, por lo que si también te ha salido en rojo la opción, solo deja las opciones predeterminadas.

Luego de configurar instalaremos el sistema operativo. Una vez terminada la instalación iniciarás sesión con el usuario creado anteriormente, y con esto ya estará listo Fedora Server para ser utilizado.

4. Creación de script de suba y baja de usuarios y grupos

Diseñamos un script modulado para gestionar usuarios y grupos.

Su funcionamiento consta de 2 scripts: uno llamado menuPrincipal.sh, donde se corroborará la autenticación del usuario y del acceso a las pantallas de gestor de usuarios y grupos en caso de que el usuario sea aceptado, y otro llamado gestiones.sh, que se encargará de realizar la función de alta y baja de los usuarios y grupos.



4.1 Menú principal

```
#!/bin/bash
if [ $EUID -eq 0 ]; then
    source ./gestiones.sh
    opc=0
    while (($opc !=3)); do
        clear
        echo ":::::::::Usuarios y grupos:::::::::"
        echo "*****"
        echo ""
        echo "1. Gestión de usuarios"
        echo "-----"
        echo "2. Gestión de grupos"
        echo "-----"
        echo "3. Salir"
        echo "-----"
        read -p "Eliga una opción:" opc
        case $opc in
            1) menuUsuario;;
            2) menuGrupo;;
            3) echo "Saliendo..."
               read -p "Presione ENTER para salir";;
            *) echo "Opción no válida"
               read -p "Presione ENTER para continuar";;
        esac
    done
else
    echo "(!) Se requieren privilegios de superusuario (!)"
fi
```

Este script es la entrada principal al gestor de usuarios y grupos, su función principal es verificar si el usuario quien está intentando ejecutar el programa forma parte del grupo 'wheel'.

Para lograr el resultado se consideró la variable de entorno 'EUID', que verifica si el usuario activo tiene privilegios de superusuario con valores binarios.

Sabiendo que el valor 0 determina que el usuario en cuestión tiene privilegios del grupo wheel se diseña un if cuya condición sea que la variable 'EUID' tenga asignado el valor 0, ya que cuando un usuario con privilegios ejecuta un comando utilizando 'sudo', su valor cambia por un segundo a 0, el Identificador de usuario de la cuenta root.

Si la condición anterior no se cumple, se imprimirá un mensaje de error: "Se requieren privilegios de de superusuario".

Si la condición anterior se cumple, se extraerá la información del script gestiones.sh mediante el comando 'source', esto con el objetivo de ejecutar las funciones que se encuentran dentro de este script en el script menuPrincipal.sh

Continuando con la condición verdadera, se realizará un menú de opciones, para esto creamos la variable 'opc' para poder realizar la condición de una estructura while para el

CEITAROX

I.S.B.O.

3MJ



menú. Este while se crea con la intención de que el menú esté en bucle hasta que la variable 'opc' sea de valor 3 (opción de "Salir" en el menú). Mediante el comando 'read' junto con el delimitador -p para mostrar un mensaje de solicitud al usuario de elegir entre la opción 1 "Gestión de usuarios", la opción 2 "Gestión de grupos" y la opción 3 ya mencionada "Salir", la respuesta dada al comando será almacenada en la variable 'opc'. Con el comando 'case' se utilizará el valor de la variable dado por el comando 'read' para ejecutar las funciones del menú del script gestiones.sh para el caso 1 y 2, el caso 3 hará cerrar el ciclo while y, por ende, el programa. Si se encuentra un valor distinto a 1, 2 o 3 se imprimirá un mensaje default "Opción no válida", y luego de dar ENTER se reiniciará el menú de opciones gracias al 'while'.



4.2 Menú de gestión de usuarios y grupos

```
#!/bin/bash

function menuUsuario(){

    local opc=0
    while (($opc != 3)); do

        clear
        echo ":::::Bienvenido al menú de usuarios:::::"
        echo "*****"
        echo "1. Alta usuario"
        echo "-----"
        echo "2 Baja usuario"
        echo "-----"
        echo "3. Volver al menú principal"
        echo "-----"
        read -p "Ingrese una opción:" opc

        case $opc in
            1)altaUsuario;;
            2)bajaUsuario;;
            3)return;;
            *)echo "Opción no válida"
              read -p "Presione ENTER para continuar";;
        esac

    done
}

function altaUsuario(){

    clear
    read -p "Ingrese el nombre del usuario: " usuario
    read -p "Ingrese el directorio de home:" directorio

    if grep -q "^$usuario:" /etc/passwd; then
        echo "El usuario $usuario ya existe"
    else
        useradd -d "/home/$directorio" -m -s /bin/bash "$usuario"
        echo "$usuario:$usuario" | chpasswd
        chage -d 0 "$usuario"
        echo "Se he creado el usuario $usuario en /home/$directorio con contraseña temporal $usuario"
    fi

    read -p "Presione ENTER para continuar"
}

function bajaUsuario(){
```

Función menuUsuario():

Se crea una variable local 'opc' con valor 0, y al igual que el menú de gestión de menuPrincipal.sh, se realiza un 'while' para generar un bucle para este nuevo menú de gestión de usuario.

A comparación del menú principal las opciones pasan a ser: opción 1 "Alta usuario", donde como resultado se ejecutará la función altaUsuario(). Opción 2 "Baja usuario", donde como respuesta se ejecuta la función bajaUsuario(). Por último, opción 3 "Volver al menú principal", donde se ejecuta el comando 'return' para regresar al script anterior. También se aplica una opción default con mensaje de error si los valores no son ni 1, 2 o 3.



Función altaUsuario():

Luego de limpiar la pantalla utilizando 'clear' para más comodidad visual, se le pide al usuario mediante dos 'read' ingrese el nombre de usuario a crear y el nombre del directorio de trabajo home, las respuestas se almacenarán en las variables 'usuario' y 'directorio' respectivamente.

Se verifica con un 'grep' si el usuario ya existe, para esto se revisa su existencia en el directorio '/etc/passwd', donde se guarda la información de los usuarios, incluyendo su nombre. Si la condición es verdadera, se imprime un mensaje indicando que el usuario ya existe. Si la condición es falsa, entonces se ejecutará el comando 'useradd' para crear usuarios, donde le daremos la habilitación del uso del lenguaje Bash y la dirección del directorio home del nuevo usuario utilizando la variable 'directorio' al final de la dirección. Luego de crear la carpeta ('-m') e indicar el shell ('-s'), colocamos la variable 'usuario' para mostrarle al comando el nombre del nuevo usuario.

Creamos una contraseña temporal, para esto utilizamos el 'chpasswd', indicándole que la contraseña del usuario nuevo es su nombre. Con la sintaxis 'chage -d 0' forzamos al usuario a cambiar la contraseña en el momento que inicie sesión. Luego de esto imprimimos un mensaje indicando que se creó el nuevo usuario con respectivo directorio y contraseña temporal exitosamente

.



```
clear
read -p "Ingrese el nombre del usuario a eliminar: " usuario
read -p "¿Desea eliminar también el directorio de home? (y/n): " caso

if [[ $caso == "y" ]]; then
    userdel -r "$usuario"
    echo "Se ha eliminado el usuario $usuario y su directorio home"
elif [[ $caso == "n" ]]; then
    userdel "$usuario"
    echo "Se ha eliminado el usuario $usuario"
else
    echo "Se canceló la baja de usuario por un error en la opción ingresada"
fi

read -p "Presione ENTER para continuar"

}

function menuGrupo(){
    local opc=0
    while (( $opc != 3 )); do
        clear
        echo ":::::Bienvenido al menú de grupos:::::"
        echo "*****"
        echo " "
        echo "1. Alta grupo"
        echo "-----"
        echo "2. Baja grupo"
        echo "-----"
        echo "3. Volver al menú principal"
        echo "-----"
        read -p "Ingrese una opción: " opc

        case $opc in
            1)altaGrupo;;
            2)bajaGrupo;;
            3)return;;
            *) echo "Opción no válida"
              read -p "Presione ENTER para continuar" ;;
        esac
    done
}
```

Función bajaUsuario():

Para realizar la baja de usuario, utilizamos un 'read' para pedirle al usuario el nombre del usuario a eliminar, almacenando el nombre en la variable 'usuario'. También se consulta al usuario si se desea eliminar también el directorio home del usuario a eliminar, con las opciones y(yes) o n(no), la respuesta almacenandose en la variable 'caso'. Se crea una condicional para la variable 'caso', si la opción es 'y' entonces se ejecutará el comando 'userdel' y se indicará el nombre de usuario a eliminar con la variable 'usuario', para que se elimine también su directorio se utiliza '-r'. Si la respuesta es 'n', se ejecuta el mismo comando sin el '-r'. Si no se ejecuta ninguna de esas dos opciones, se imprime un mensaje de error default.

Función menuGrupo():

CEITAROX**I.S.B.O.****3MJ**



Se realiza un menú igual a la función `menuUsuario()`, donde lo único que cambia son las opciones de alta y baja de usuario, las cuales pasan a ser de alta y baja de grupos. Si se elige la opción 1, se ejecutará la función `altaGrupo()`, si se elige la opción 2, se ejecuta la función `bajaGrupo()`.

```
2. Baja grupo"
3. Volver al menú principal"
read -p "Ingrese una opción: " opc

case $opc in
    1)altaGrupo;;
    2)bajaGrupo;;
    3)return;;
    *) "Opción no válida"
      read -p "Presione ENTER para continuar" ;;
esac

done
}

function altaGrupo(){
    read -p "Ingrese el nombre del grupo: " grupo
    if [[ -z "$grupo" ]]; then
        echo "Nombre de grupo no válido"
    else
        groupadd "$grupo"
        echo "Se ha creado el grupo $grupo"
    fi
    read -p "Presione ENTER para continuar"
}

function bajaGrupo(){
    read -p "Ingrese el nombre del grupo a eliminar: " grupo
    if [[ -z "$grupo" ]]; then
        echo "Nombre de grupo no válido"
    else
        sudo groupdel "$grupo"
        echo "Se eliminó el grupo $grupo"
    fi
    read -p "Presione ENTER para continuar"
}
```

Función `altaGrupo()`:

Se le pide al usuario, con 'read' como las anteriores veces, que ingrese el nombre del grupo nuevo, la información guardándose en la variable 'grupo'. Se realiza una condicional para verificar que la variable 'grupo' no esté vacía, si la variable está vacía se imprime un mensaje de error indicando que el nombre no es 'válido'. Si la variable no está vacía, se llama al comando 'groupadd' y le damos la variable 'grupo', por último se muestra un mensaje indicando que el nuevo grupo fue creado con éxito.

Función `bajaGrupo()`:

CEITAROX

I.S.B.O.

3MJ



Se le pide al usuario, con 'read' como las anteriores veces, que ingrese el nombre del grupo a eliminar, la información guardándose en la variable 'grupo'. Se realiza una condicional para verificar que la variable 'grupo' no esté vacía, si la variable está vacía se imprime un mensaje de error indicando que el nombre no es 'válido'(En un futuro mejoraremos esta condicional para que verifique también si existe un grupo con el mismo nombre que la información dentro de la variable). Si la variable no está vacía, se llama al comando 'groupdel' y le damos la variable 'grupo', por último se muestra un mensaje indicando que el grupo fue eliminado con éxito.



5. Instalación de paquetes

Ya instalado el sistema operativo procedimos a agregar los paquetes Cockpit, OpenSSH-Server y NetworkManager-TUI para gestionar más adelante la configuración de red, el control remoto de parte de los integrantes y nuestra aplicación web al servidor.

```
tatiana@10:~$ dnf search networkmanager-tui
Actualizando y cargando repositorios:
Fedora 42 openh264 (From Cisco) - x86_64          100% | 2.0 KiB/s | 5.8 KiB | 00m03s
Fedora 42 - x86_64                                100% | 1.0 MiB/s | 35.4 MiB | 00m36s
Fedora 42 - x86_64 - Updates                      100% | 2.0 MiB/s | 7.5 MiB | 00m04s
Repositorios cargados.
Matched fields: name
NetworkManager-tui.x86_64: NetworkManager curses-based UI
```

Se inició la búsqueda de los paquetes por medio del comando “dnf” para investigar los repositorios necesarios y concretos para la posterior gestión comenzando por NetworkManager-TUI, el cual escogimos para la configuración IP del servidor ofreciendo una interfaz gráfica que facilita el trabajo.

```
tatiana@10:~$ sudo dnf install NetworkManager-tui
Actualizando y cargando repositorios:
Repositorios cargados.
Package Arch Version Repository Size
Installing:
NetworkManager-tui x86_64 1:1.52.0-1.fc42 fedora 711.0 KiB
Installing dependencies:
ncut x86_64 0.52.25-1.fc42 fedora 213.7 KiB
slang x86_64 2.3.3-7.fc42 fedora 1.4 MiB

Transaction Summary:
Installing: 3 packages

El tamaño total de paquetes entrantes es 775 KiB. Se necesita descargar 775 KiB.
Después de esta operación, 2 MiB extra serán utilizados (instalar 2 MiB, eliminar 0 B).
Is this ok [y/N]:
```

Mediante la modalidad de superusuario se procede a su instalación con el subcomando “install” pidiendo el sistema la autorización del superusuario que indicó el comando antes de su ejecución.



```
El tamaño total de paquetes entrantes es 775 KiB. Se necesita descargar 775 KiB.
Después de esta operación, 2 MiB extra serán utilizados (instalar 2 MiB, eliminar 0 B).
Is this ok [y/N]: y
[1/3] newt-0:0.52.25-1.fc42.x86_64 100% | 96.2 KiB/s | 114.7 KiB | 00m01s
[2/3] NetworkManager-tui-1:1.52.0-1.fc42.x86_64 100% | 155.8 KiB/s | 227.4 KiB | 00m01s
[3/3] slang-0:2.3.3-7.fc42.x86_64 100% | 247.5 KiB/s | 433.3 KiB | 00m02s
-----
[3/3] Total 100% | 120.8 KiB/s | 775.5 KiB | 00m06s
Ejecutando transacción
[1/5] Verificar archivos de paquete 100% | 37.0 B/s | 3.0 B | 00m00s
[2/5] Preparar transacción 100% | 3.0 B/s | 3.0 B | 00m01s
[3/5] Instalando slang-0:2.3.3-7.fc42.x86_64 100% | 4.4 MiB/s | 1.4 MiB | 00m00s
[4/5] Instalando newt-0:0.52.25-1.fc42.x86_64 100% | 189.6 KiB/s | 226.9 KiB | 00m01s
[5/5] Instalando NetworkManager-tui-1:1.52.0-1.fc42.x86_64 100% | 255.3 KiB/s | 713.5 KiB | 00m03s
¡Completado!
```

Una vez autorizado se inicia su instalación.

```
tatiana@10:~$ dnf search cockpit
Actualizando y cargando repositorios:
Repositorios cargados.
Matched fields: name (exact)
cockpit.x86_64: Web Console for Linux servers
Matched fields: name, summary
cockpit-389-ds.noarch: Cockpit UI Plugin for configuring and administering the 389 Directory Server
cockpit-bridge.noarch: Cockpit bridge server-side component
cockpit-composer.noarch: Composer GUI for use with Cockpit
cockpit-doc.noarch: Cockpit deployment and developer guide
cockpit-files.noarch: A filesystem browser for Cockpit
cockpit-ha-cluster.noarch: Cockpit application for managing Pacemaker based clusters
cockpit-image-builder.noarch: Image builder plugin for Cockpit
cockpit-kdump.noarch: Cockpit user interface for kernel crash dumping
cockpit-machines.noarch: Cockpit user interface for virtual machines
cockpit-navigator.noarch: A File System Browser for Cockpit
cockpit-networkmanager.noarch: Cockpit user interface for networking, using NetworkManager
cockpit-ostree.noarch: Cockpit user interface for rpm-ostree
cockpit-packagekit.noarch: Cockpit user interface for packages
cockpit-podman.noarch: Cockpit component for Podman containers
cockpit-selinux.noarch: Cockpit SELinux package
cockpit-session-recording.noarch: Cockpit Session Recording
cockpit-sosreport.noarch: Cockpit user interface for diagnostic reports
cockpit-storaged.noarch: Cockpit user interface for storage, using udisks
cockpit-system.noarch: Cockpit admin interface package for configuring and troubleshooting a system
cockpit-ws.x86_64: Cockpit Web Service
cockpit-ws-selinux.x86_64: SELinux security policy for cockpit-ws
subscription-manager-cockpit.noarch: Subscription Manager Cockpit UI
Matched fields: summary
polarsys-b612-fonts.noarch: Sans-serif fonts designed for reading comfort and safety in aeroplane cockpits
polarsys-b612-mono-fonts.noarch: Monospace fonts designed for reading comfort and safety in aeroplane cockpits
```

En la búsqueda de repositorios para Cockpit se escogió la comentada para consola web ya que los demás repositorios ofrecen funciones no relacionadas a nuestros objetivos planteados para su mantenimiento. A su vez este repositorio ofrece la ventaja de su utilización en formato sitio web para dar soporte al estado y necesidades del servidor.

```
tatiana@10:~$ sudo dnf install cockpit
[sudo] contraseña para tatiana:
Actualizando y cargando repositorios:
Repositorios cargados.
Package "cockpit-336.2-1.fc42.x86_64" is already installed.
Nothing to do.
```

En este caso no hubo necesidad de instalarlo debido a que se instaló de forma predeterminada durante la instalación de Fedora.

CEITAROX I.S.B.O. 3MJ



```
tatiana@10:~$ dnf search openssh
Actualizando y cargando repositorios:
Repositorios cargados.
Matched fields: name (exact)
openssh.x86_64: An open source implementation of SSH protocol version 2
Matched fields: name, summary
lxqt-openssh-askpass.x86_64: Askpass openssh transition dialog for LXQt desktop suite
lxqt-openssh-askpass-l10n.x86_64: Translations for lxqt-openssh-askpass
openssh-askpass.x86_64: A passphrase dialog for OpenSSH and X
openssh-keycat.x86_64: A mls keycat backend for openssh
openssh-ldap-authkeys-selinux.noarch: SELinux module for openssh-ldap-authkeys
openssh-sk-dummy.x86_64: OpenSSH SK driver for test purposes
perl-Net-OpenSSH.noarch: Perl SSH client package implemented on top of OpenSSH
rust-openssh-keys+default-devel.noarch: Read and write OpenSSH public keys
rust-openssh-keys-devel.noarch: Read and write OpenSSH public keys
Matched fields: name
gsi-openssh.x86_64: An implementation of the SSH protocol with GSI authentication
gsi-openssh-clients.x86_64: SSH client applications with GSI authentication
gsi-openssh-keysign.x86_64: A helper program used for host-based authentication
gsi-openssh-server.x86_64: SSH server daemon with GSI authentication
openssh-clients.x86_64: An open source SSH client applications
openssh-keysign.x86_64: A helper program used for host-based authentication
openssh-ldap-authkeys.noarch: Python script to generate SSH authorized_keys files using an LDAP directory
openssh-server.x86_64: An open source SSH server daemon
Matched fields: summary
keychain.noarch: Agent manager for OpenSSH, ssh.com, Sun SSH, and GnuPG
rssh.x86_64: Restricted shell for use with OpenSSH, allowing only scp and/or sftp
x11-ssh-askpass.x86_64: A passphrase dialog for X and not only for OpenSSH

tatiana@10:~$ sudo dnf install openssh
[sudol contraseña para tatiana:
Actualizando y cargando repositorios:
Repositorios cargados.
Package "openssh-9.9p1-10.fc42.x86_64" is already installed.

Nothing to do.
```

Lo mismo ha trascendido con el montaje de OpenSSH-Server, el cual será utilizado para el control remoto del servidor.

Se piensa próximamente agregar el repositorio Docker, el cual tendrá como misión la conexión de la base de datos con el servidor y almacenar de forma indefinida la información almacenada en esta siendo autorizado su ingreso o eliminación por parte del usuario final.



6. Revisión del firewall, rich rules e iptables

Una vez verificado que el servicio SSH está instalado en el sistema operativo se hacen más procesos de observación al resto del entorno de Fedora. En esta carpeta sólo nos centraremos en el firewall, las rich rules y las iptables con el objetivo de determinar si SSH está totalmente habilitado en la máquina virtual para próximamente iniciar las conexiones remotas.

```
tatiana@10:~$ systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf, 50-keep-warm.conf
   Active: active (running) since Sat 2025-07-12 15:46:23 -03; 1h 31min ago
   Invocation: d3f606b1065644579506a691eecc049
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 833 (sshd)
     Tasks: 1 (limit: 2302)
    Memory: 1.4M (peak: 1.6M)
       CPU: 217ms
   CGroup: /system.slice/ssh.service
           └─833 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

jul 12 15:46:19 localhost.localdomain systemd[1]: Starting sshd.service - OpenSSH server daemon...
jul 12 15:46:23 localhost.localdomain sshd(833): Server listening on 0.0.0.0 port 22.
jul 12 15:46:23 localhost.localdomain sshd(833): Server listening on :: port 22.
jul 12 15:46:23 localhost.localdomain systemd[1]: Started sshd.service - OpenSSH server daemon.
```

Se inició revisando si el sistema lo estaba bloqueando con el uso del comando 'systemctl', el cual se encarga de gestionar los servicios y demonios del mismo sistema operativo, y el subcomando 'status', que se utiliza para listar servicios. Esto al combinarlo con el nombre del servicio SSH ('sshd') dió como resultado la información de su estado actual indicando de que estaba habilitado, para una mejor corroboración se reinició el servicio usando el subcomando 'restart'.

En caso de que no estuviera habilitado se iniciaría el servicio con la sentencia 'systemctl enable sshd'.

```
tatiana@10:~$ sudo firewall-cmd --list-services
[sudo] contraseña para tatiana:
cockpit dhcpv6-client ssh
```

Ya comprobado el sistema se inicia analizando el cortafuegos con privilegios de superusuario con una sentencia parecida pero diferente de la anterior usando el comando



‘firewall-cmd’, que es lo mismo que systemctl pero destinado para los protocolos de red e IPs, con el delimitador ‘--list-services’, que lista los protocolos habilitados para utilizar en la máquina, obteniendo la confirmación de que el servicio SSH está también autorizado para su uso en la red.

Aún así para confirmarlo en su totalidad se han revisado las rich rules e iptables, ya que forman parte de la jerarquía del comando ‘firewall-cmd’ y son las que realmente impiden el correcto funcionamiento de ciertos protocolos de internet, como SSH.

```
tatiana@10:~$ sudo iptables -L | grep ssh  
[sudo] contraseña para tatiana:
```

```
tatiana@10:~$ sudo firewall-cmd --list-rich-rules  
tatiana@10:~$
```

La primer sentencia busca en su lista de IPs (delimitador -L) la IP con nombre SSH.

La segunda utiliza nuevamente el comando ‘firewall-cmd’ busca directamente en las rich rules si se encuentra un protocolo.

En ambas sentencias no se halló ningún protocolo.

7. Configuración del protocolo SSH

Por último para la total definición de este puerto se van a modificar algunas propiedades del mismo, más específicamente el ingreso como root al servidor, la autenticación por contraseña (incluido por teclado) y la autenticación por clave pública, queremos a su vez modificar esas propiedades para iniciar la aplicación de un nuevo sistema de seguridad por medio de las claves público-privada optimizando considerablemente el cifrado de las transferencias de datos en el servidor.

Las mencionadas propiedades se encuentran en uno o más archivos ‘.conf’ dentro de este directorio: /etc/ssh, por lo que se tuvieron que analizar los archivos de esa ruta.



Una vez llegada a la ruta se encontraron 2 archivos relacionados a específicamente su configuración siendo sshd_config el principal (el que abarcaba todas las preferencias) y ssh_config el secundario.

En ssh_config;

```
# Configuration data is parsed as follows:
# 1. command line options
# 2. user-specific file
# 3. system-wide file
# Any configuration value is only changed the first time it is set.
# Thus, host-specific definitions should be at the beginning of the
# configuration file, and defaults at the end.

# Site-wide defaults for some commonly used options. For a comprehensive
# list of available options, their meanings and defaults, please see the
# ssh_config(5) man page.

# Host *
# ForwardAgent no
# ForwardX11 no
# PasswordAuthentication yes
# HostbasedAuthentication no
# GSSAPIAuthentication no
# GSSAPIDelegateCredentials no
# GSSAPIKeyExchange no
# GSSAPITrustDNS no
# BatchMode no
# CheckHostIP no
# AddressFamily any
# ConnectTimeout 0
# StrictHostKeyChecking ask
# IdentityFile ~/.ssh/id_rsa
# IdentityFile ~/.ssh/id_dsa
# IdentityFile ~/.ssh/id_ecdsa
# IdentityFile ~/.ssh/id_ed25519
# Port 22
# Ciphers aes128-ctr,aes192-ctr,aes256-ctr,aes128-cbc,3des-cbc
# MACs hmac-md5,hmac-sha1,umac-64@openssh.com
# EscapeChar ~
# Tunnel no
# TunnelDevice any:any
# PermitLocalCommand no
# VisualHostKey no
# ProxyCommand ssh -q -W %h:%p gateway.example.com
# RekeyLimit 1G 1h
# UserKnownHostsFile ~/.ssh/known_hosts.d/%k

# This system is following system-wide crypto policy.
# To modify the crypto properties (Ciphers, MACs, ...), create a *.conf
# file under /etc/ssh/ssh_config.d/ which will be automatically
# included below. For more information, see manual page for
# update-crypto-policies(8) and ssh_config(5).
include /etc/ssh/ssh_config.d/*.conf
'ssh_config' [Sólo lectura] 55L, 1916B
```

55,29

Final

No hubo necesidad de modificación ya que la propiedad 'PasswordAuthentication' no perjudicó a la próxima implementación de la clave pública-privada.



En sshd_config:

```
#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# The default is to check both .ssh/authorized_keys and .ssh/authorized_keys2
# but this is overridden so installations will only check .ssh/authorized_keys
AuthorizedKeysFile .ssh/authorized_keys

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
#PasswordAuthentication no
#PermitEmptyPasswords no

# Change to no to disable s/key passwords
#KbdInteractiveAuthentication no
```

Se encontraron las propiedades a buscar ('PermitRootLogin', 'PubkeyAuthentication', 'PasswordAuthentication' y 'KbdInteractiveAuthentication') y se modificaron todas ellas.

La que corresponde al ingreso como root al servidor ('PermitRootLogin') se la estableció a "prohibit-password", que restringe al usuario root el ingreso al servidor por protocolo SSH pero sin prohibirle su acceso, debido a que root es una cuenta muy poderosa e importante que debe ser protegida para evitar su uso de cualquier usuario corriente, y las propiedades 'PasswordAuthentication' y 'KbdInteractiveAuthentication' ya que consideramos la contraseña como un modelo de autenticación débil e inseguro para un servidor que almacenará información confidencial como el código fuente y la base de datos de la aplicación.



A la propiedad 'PubkeyAuthentication' se le asignó la opción "yes" ya que esa característica será fundamental para permitir establecer el sistema de claves público-privada.

8. Implementación de clave pública-privada

Nosotros hemos decidido implementar un nuevo sistema de autenticación para el ingreso al servidor. Para su aplicación nuestro plan es crear una clave pública y una clave privada y luego de crearlas trasladar la clave pública al servidor por medio del protocolo SSH.

La función que cumplirá la clave privada es ser la "llave de acceso" al servidor mientras que la clave pública verifica que la clave privada contenga el patrón principal que comparten ambas llaves al ser generadas.

```
Administrador: Símbolo del sistema
Microsoft Windows [Versión 10.0.26100.4652]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\57136529\.ssh/id_ed25519):
Created directory 'C:\Users\57136529\.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\57136529\.ssh/id_ed25519
Your public key has been saved in C:\Users\57136529\.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:wLfSMpKBLrGbNYoJ3c9dW6fJ4+0UBoy7nls4C2Q+UMA ceibal-prod\57136529@T140J2401081963
The key's randomart image is:
+--[ED25519 256]--+
|      .E.  o      |
|    . . . O O . o  |
|  = . O = . . .   |
|+ = + = S O . +   |
|oB . + X . * = .   |
|*      o = * .     |
|      + * +        |
|     =...O         |
+----[SHA256]-----+

C:\Windows\System32>
```

La generación de las claves se realizó con la interfaz de comando de Windows con el comando 'ssh-keygen' creándose por la arquitectura por defecto ('ED25519'). Durante el procedimiento el sistema nos permitió agregar una contraseña la cual llamamos "1F9l8@3wEr" con el propósito de convertir la clave

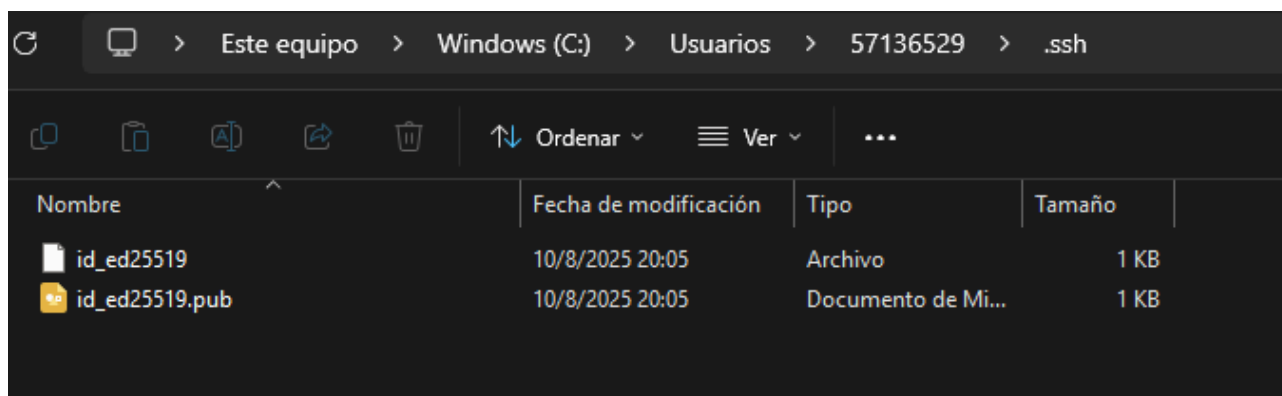
CEITAROX

I.S.B.O.

3MJ



pública-privada en un sistema de doble autenticación, por un lado la existencia de la clave privada y por otro el acierto de la contraseña para ingresar al servidor.



Una vez que las claves se crearon Windows las almacena en esta dirección:

C:\Users\nombre_del_usuario_quien_las_crea\.ssh

Ambas claves se distinguen por una de ellas que tiene una extensión siendo la clave pública definida como el formato '.pub'.

8.1 Conectividad de la máquina física con el servidor

Antes de trasladar la clave pública al servidor tuvimos que resolver la cuestión de cómo conseguir la conectividad de una tarjeta de red de nuestros equipos a la máquina virtual. Dado que luego de la segunda entrega debemos presentar el funcionamiento del servidor en la institución tuvimos que tener en consideración el funcionamiento de la red de Plan Ceibal.

Nos dimos cuenta que Plan Ceibal no tolera la conexión de máquinas virtuales a su red por medio del "modo adaptador puente", esta conexión trata de que VMware "llama" al router de la máquina física y genera una IP a la máquina virtual para que forme parte de la red funcionando como un dispositivo independiente, pero lo reconoce como una actividad sospechosa. Por esta razón vimos necesario utilizar otro método de conexión siendo escogida la conexión "Host-Only" o "Sólo Arfitrión", consiste en crear una red



privada de la máquina virtual con el equipo físico que la almacena creando en la máquina virtual una tarjeta de red ficticia para su conexión, esto proporciona una conexión segura y funcional aislada de la red de Plan Ceibal impidiendo la conexión inversa de la máquina virtual a la máquina física.

8.2 Testeo de conexión

```
tatiana@localhost:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: emp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:7e:c3:0c brd ff:ff:ff:ff:ff:ff
    altname enx0800277ec30c
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute emp0s3
        valid_lft 356sec preferred_lft 356sec
    inet6 fe80::a00:27ff:fe7e:c30c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

La tarjeta que genera VMware es la nombrada como “emp0s3” con la dirección IP 192.168.56.101.

```
C:\Users\57136529>ping 192.168.56.101

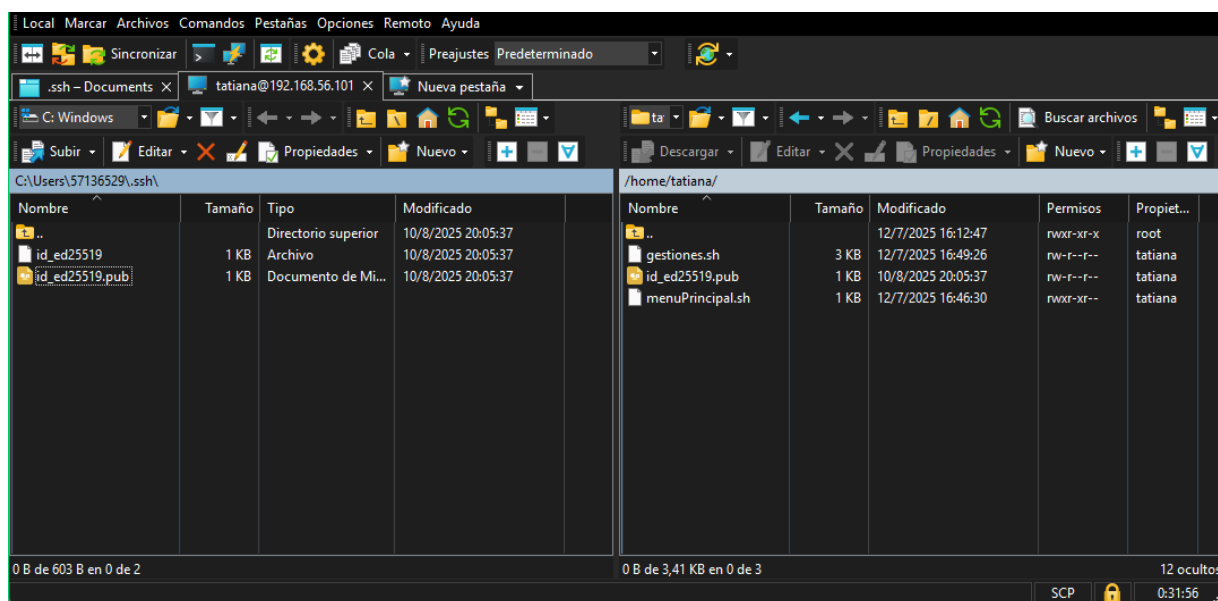
Haciendo ping a 192.168.56.101 con 32 bytes de datos:
Respuesta desde 192.168.56.101: bytes=32 tiempo=3ms TTL=64
Respuesta desde 192.168.56.101: bytes=32 tiempo=1ms TTL=64
Respuesta desde 192.168.56.101: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.56.101: bytes=32 tiempo=3ms TTL=64

Estadísticas de ping para 192.168.56.101:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 0ms, Máximo = 3ms, Media = 1ms
```

Al comprobar la conexión desde la máquina física se observa que existe interacción y conectividad,



8.3 Traslado y adaptación de la clave pública a el servidor



Ya solucionadas las dificultades de conectividad procedimos a utilizar el protocolo SSH con el comando SCP para trasladar la clave pública al servidor mediante el programa WinSCP para no escribir comando.

Dado que en Fedora no se ha generado ninguna clave pública-privada el sistema no ha creado un directorio para almacenarlas.

```
AuthorizedKeysFile .ssh/authorized_keys
```

Si observamos nuevamente el archivo 'sshd_config' se muestra que directorio que se define al crearlas es .ssh/authorized_keys dentro de /home/nombre_usuario, es en ese directorio donde únicamente funcionarán las claves.



```
tatiana@localhost:~$ mkdir .ssh
tatiana@localhost:~/.ssh$ cd .ssh
tatiana@localhost:~/.ssh$ touch authorized_keys
tatiana@localhost:~/.ssh$ cd ..
tatiana@localhost:~$ cat id_ed25519.pub > authorized_keys
tatiana@localhost:~$
```

Creado el directorio se extrae el contenido de la clave pública y se inserta en el archivo 'authorized_keys'.

8.4 Testeo final del sistema de autenticación

```
Microsoft Windows [Versión 10.0.26100.4652]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\57136529>ssh tatiana@192.168.56.101
Enter passphrase for key 'C:\Users\57136529\.ssh\id_ed25519':
Enter passphrase for key 'C:\Users\57136529\.ssh\id_ed25519':
Enter passphrase for key 'C:\Users\57136529\.ssh\id_ed25519':
tatiana@192.168.56.101's password:
Web console: https://localhost:9090/

Last login: Sun Aug 10 22:41:05 2025
tatiana@localhost:~$ |
```

Ya puesta la clave pública acorde a RedHat en la máquina virtual se realizó un testeo final desde la máquina física para comprobar si este sistema de autenticación funciona correctamente y permite el acceso mostrando como resultado un rendimiento satisfactorio.



9. Continuación del script (creación de las funciones de firewalld y respaldo)

Para la segunda entrega se hicieron modificaciones importantes a la modulación de los scripts dividiendo el menú principal del script y el de todas sus funciones por separado, siendo definidas de esta manera:

menuPrincipal.sh -> Menú principal del script

menuUsuario.sh -> Menú principal para la gestión de usuarios

menuGrupo.sh -> Menú principal para la gestión de grupos

También se agregaron 2 nuevas funciones, las cuales son un menú de gestión de firewalld y un menú para administrar respaldos. Para estas dos funciones hicieron falta diseñar 3 scripts siendo nombrados:

menuFirewall.sh -> Menú principal para la administración de firewall

gestionZonas.sh -> Submenú de firewall encargado de la gestión de zonas

menuRespaldos.sh -> Menú principal para la administración de firewall

9.1 Firewall

Como mencionamos anteriormente la función de firewall ha conllevado de 2 scripts para su funcionamiento. Para el script menuFirewall.sh se tuvieron que utilizar 11 funciones y para gestionZonas.sh 7 funciones.

Dado que algunas de las funciones tiene una larga extensión se adjunta un anexo al script completo desde un repositorio público en GitHub.

9.1.1 menuFirewall.sh

Menú principal (función mFirewall()):

CEITAROX

I.S.B.O.

3MJ



```
function mFirewall(){
    local $opc=0
    while (($opc !=5)); do
        clear
        echo ":::::Bienvenido al menú de Firewall:::::"
        echo "*****"
        echo " "
        echo "1. Gestión de zonas"
        echo "-----"
        echo "2. Permitir/Bloquear un servicio o puerto"
        echo "-----"
        echo "3. Permitir/Bloquear tráfico según IP"
        echo "-----"
        echo "4. Permitir/Bloquear IP + puerto"
        echo "-----"
        echo "5. Volver al menú principal"
        echo "-----"
        read -p "Ingrese una opción: " opc
        case $opc in
            1)
                source ./gestionZonas.sh
                mZonas
                ;;
            2)gestionServicios;;
            3)gestionIP;;
            4)gestionIPServicio;;
            5)source ./menuPrincipal.sh;;
            *)echo "Opción no válida"
            read -p "Presione ENTER para continuar" ;;
        esac
    done
}
```

Esta función abre un case permitiendo el acceso al resto de funciones internas y externas al script. En casi todas las opciones hay otros submenú para mejor variedad de aplicación. El menú consta de las siguientes opciones:

Opción 1: Gestión de zonas

Opción 2: Permitir/bloquear servicio/puerto

Opción 3: Permitir/bloquear tráfico según IP

Opción 4: Permitir/bloquear IP + puerto

Opción 5: Volver al menú principal

Función gestionServicios()



```
function gestionServicios(){
    clear
    local opc=0
    echo ":::::PERMITIR/BLOQUEAR UN SERVICIO/PUERTO:::::"
    echo ""
    echo "Todos los servicios:"
    echo ""
    firewall-cmd --get-services
    echo ""
    echo "1. Permitir un servicio"
    echo "-----"
    echo "2. Bloquear un servicio"
    echo "-----"
    echo "3. Permitir un puerto"
    echo "-----"
    echo "4. Bloquear un puerto"
    echo "-----"
    echo "5. Volver al menú principal de zonas"
    echo "-----"
    read -p "Ingrese una opción: " opc
    case $opc in
        1)permitirServicio;;
        2)bloquearServicio;;
        3)permitirPuerto;;
        4)bloquearPuerto;;
        5)mFirewall;;
    |
```

Es otro case que contiene todas las combinaciones posibles para función “Permitir/bloquear servicio/puerto”.

Función permitirServicio():

```
read -p "Ingrese el nombre del servicio: " servicio
echo ""
read -p "Desea establecerla de forma permanente? (y/n). En caso de que escoja 'n' el cambio será temporal: " caso
#Esta condicional se encuentra en todas las funciones de firewall y es importante que esté
if [[ $caso == "y" ]];then
    read -p "Desea hacerlo para una zona en particular? En caso que escoja 'n' la habilitación se hará en la zona por defecto (y/n): " op
    echo " "
    B
    if [[ $op == "y" ]];then
        read -p "Ingrese la zona a permitir: " zona
        B
        #Acción permitir servicio a zona específica de forma permanente (no deshace al reiniciar)
        firewall-cmd --permanent --zone="$zona" --add-service="$servicio"
        B
        echo " "
        read -p "Presione ENTER para volver al menú principal de zonas"
    elif [[ $op == "n" ]]; then
        #Acción permitir servicio a zona por defecto de forma permanente
        firewall-cmd --permanent --add-service="$servicio"
        echo " "
        read -p "Presione ENTER para volver al menú principal de zonas"
    else
        echo " "
        echo "Se canceló la acción por un error en la opción ingresada"
    fi
fi
-- INSERTAR --
```

95,4-25



Esta función ejecuta un if anidado que solicita los datos de la zona y el servicio con 'read -p' guardándolos en variables para aplicarlos al comando de 'firewall-cmd --add-service --zone'.

Antes de solicitar estos datos se le pregunta al usuario si desea aplicar estos cambios de forma permanente (resistente al reinicio pero no se aplica de inmediato) o temporalmente (se borran los cambios una vez reiniciado el equipo). Si el usuario responde con "y" se le agrega a la sentencia de 'firewall-cmd --add-service=\$servicio --zone=\$zona' el delimitador '--permanent', si responde "n" no se le agrega ningún delimitador.

Función bloquearServicio():

```
function bloquearServicio(){
    read -p "Ingrese el nombre del servicio: " servicio
    echo ""
    read -p "Desea establecerla de forma permanente? (y/n). En caso de que escoja 'n' el cambio será temporal: " caso
    if [[ $caso == "y" ]];then
        read -p "Desea hacerlo para una zona en particular? En caso que escoja 'n' la deshabilitación se hará en la zona por defecto (y/n): " op
        echo " "
        if [[ $op == "y" ]];then
            read -p "Ingrese la zona a bloquear: " zona
            firewall-cmd --permanent --zone="$zona" --remove-service="$servicio"
            echo ""
            read -p "Presione ENTER para volver al menú principal de zonas"
        elif [[ $op == "n" ]]; then
            firewall-cmd --permanent --remove-service="$servicio"
            echo ""
            read -p "Presione ENTER para volver al menú principal de zonas"
        else
            echo ""
            echo "Se canceló la acción por un error en la opción ingresada"
        fi
    fi
}
```

Hace lo mismo que la función permitirServicio() solo que en vez de utilizar el comando 'firewall-cmd --add-service --zone' hace 'firewall-cmd --remove-service --zone'.



Función permitirPuerto():

```
function permitirPuerto(){
    clear
    echo ""
    read -p "Desea establecerla de forma permanente? (y/n). En caso de que escoja 'n' el cambio será temporal: " caso
    if [[ $caso == "y" ]];then
        read -p "Desea hacerlo para una zona en particular? En caso que escoja 'n' la habilitación se hará en la zona por defecto (y/n): " op
        echo " "
        if [[ $op == "y" ]]; then
            read -p "Escriba la zona que desea asignar:" zona
            echo " "
            #Crecimiento de condicionales
            read -p "Quiere asignar un rango de puertos? (y/n): " o
            if [[ $o == "y" ]];then
                echo " "
                read -p "Ingrese el primer parámetro: " puertominimo
                echo " "
                read -p "Ingrese el segundo parámetro: " puertomaximo
                #El for recorre la variable de "puertominimo" hasta llegar al valor de "puertomaximo"
                for (( i=puertominimo; i <= puertomaximo; i++ )); do
                    firewall-cmd --permanent --zone="$zona" --add-port="$i"/tcp
                done
            else
                echo " "
            fi
        else
            echo " "
        fi
    fi
}
```

Función que cambia el uso de servicios con el de los puertos. Se agrega otra petición extra que le pregunta al usuario si quiere asignar un rango de puertos. En caso de que se escoja la opción mencionada se le solicita al usuario el inicio y final de los puertos y se realiza un for con dichos parámetros donde la variable "i" representa el puerto mínimo ingresado por el usuario.

9.1.2 gestionZonas.sh

Menú principal (función mZonas()):

Esta función abre un case permitiendo el acceso al resto de funciones internas y externas al script. En casi todas las opciones hay otros submenú para mejor variedad de aplicación. El menú consta de las siguientes opciones:

Opción 1: Asignar interfaz/puerto de red a zona

Opción 2: Asignar interfaz/puerto de red a zona por defecto

Opción 3: Listar zonas

CEITAROX

I.S.B.O.

3MJ



Opción 4: Cambiar zona por defecto

Opción 5: Ver el estado de un servicio

Opción 6: Volver al menú principal

9.2 Respaldos

10. Instalación de Docker Engine

Durante todo el procedimiento de instalación y uso de Docker se tuvo que utilizar el método de conexión “modo de adaptador puente”, solo para la interacción del servidor con el repositorio e imágenes de Docker y desde el domicilio de uno de los integrantes, y escribir comandos como superusuario. En aquellos momentos que no fuera necesario estas conexiones se lo dejó en la conexión “Sólo anfitrión”. Los pasos de instalación los seguimos de la página oficial de Docker, donde se detallan los pasos a seguir dependiendo del sistema operativo utilizado, incluido Fedora.

```
tatiana@192:~$ sudo dnf -y install dnf-plugins-core
[sudo] contraseña para tatiana:
Actualizando y cargando repositorios:
Fedora 42 - x86_64 - Updates
Fedora 42 openh264 (From Cisco) - x86_64
Fedora 42 - x86_64
Fedora 42 - x86_64 - Updates
Repositorios cargados.

```

Package	Arch	Version	Repository	Size
Installing:				
dnf-plugins-core	noarch	4.10.1-1.fc42	fedora	22.7 KiB
Installing dependencies:				
python3-dateutil	noarch	1:2.8.2-17.fc42	fedora	876.2 KiB
python3-dnf-plugins-core	noarch	4.10.1-1.fc42	fedora	1.2 MiB

```
Transaction Summary:
Installing: 3 packages
El tamaño total de paquetes entrantes es 714 KiB. Se necesita descargar 714 KiB.
Después de esta operación, 2 MiB extra serán utilizados (instalar 2 MiB, eliminar 0 B).
(1/3) dnf-plugins-core-0:4.10.1-1.fc42.noarch
(2/3) python3-dnf-plugins-core-0:4.10.1-1.fc42.noarch
(3/3) python3-dateutil-1:2.8.2-17.fc42.noarch
(3/3) Total
Ejecutando transacción
(1/5) Verificar archivos de paquete
(2/5) Preparar transacción
(3/5) Instalando python3-dateutil-1:2.8.2-17.fc42.noarch
(4/5) Instalando python3-dnf-plugins-core-0:4.10.1-1.fc42.noarch
(5/5) Instalando dnf-plugins-core-0:4.10.1-1.fc42.noarch
¡Completado!
```

```
tatiana@192:~$ sudo dnf -3 config-manager --add-repo https://download.docker.com/linux/fedora/docker-ce.repo
Agregando repositorio de: https://download.docker.com/linux/fedora/docker-ce.repo
```

Antes de instalarlo se tuvo que descargar el paquete “dnf-plugins-core” para después configurar el repositorio de Docker siguiendo las pautas proporcionadas del manual de

CEITAROX I.S.B.O. 3MJ



instalación con el comando 'dnf -y install dnf-plugins-core' y 'dnf -3 config-manager --add-repo <https://download-docker.com/linux/fedora/docker-ce.repo>'.

```
tatiana@192:~$ sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Actualizando y cargando repositorios:
  Docker CE Stable - x86_64
Repositorios cargados.
Package Arch Version Repository Size
Upgrading:
selinux-policy noarch 42.6-1.fc42 updates 31.6 KiB
replacing selinux-policy noarch 41.34-1.fc42 anaconda 31.4 KiB
selinux-policy-targeted noarch 42.6-1.fc42 updates 18.6 MiB
replacing selinux-policy-targeted noarch 41.34-1.fc42 anaconda 18.5 MiB
Installing:
containerd.io x86_64 1.7.27-3.1.fc42 docker-ce-stable 155.4 MiB
docker-buildx-plugin x86_64 0.26.1-1.fc42 docker-ce-stable 74.2 MiB
docker-ce x86_64 3:28.3.3-1.fc42 docker-ce-stable 86.0 MiB
docker-ce-cli x86_64 1:28.3.3-1.fc42 docker-ce-stable 35.3 MiB
docker-compose-plugin x86_64 2.39.1-1.fc42 docker-ce-stable 73.0 MiB
Installing dependencies:
container-selinux noarch 4:2.241.0-1.fc42 updates 72.6 KiB
fuse-common x86_64 3.16.2-5.fc42 fedora 38.0 B
fuse-overlayfs x86_64 1.13-3.fc42 fedora 135.0 KiB
fuse3 x86_64 3.16.2-5.fc42 fedora 129.7 KiB
libcgroup x86_64 3.0-8.fc42 fedora 157.7 KiB
libslirp x86_64 4.8.0-3.fc42 fedora 151.3 KiB
slirp4netns x86_64 1.3.1-2.fc42 fedora 89.3 KiB
Installing weak dependencies:
docker-ce-rootless-extras x86_64 28.3.3-1.fc42 docker-ce-stable 11.2 MiB
Transaction Summary:
Installing: 13 packages
Upgrading: 2 packages
Replacing: 2 packages
El tamaño total de paquetes entrantes es 115 MiB. Se necesita descargar 115 MiB.
Después de esta operación, 436 MiB extra serán utilizados (instalar 454 MiB, eliminar 19 MiB).
Is this ok [y/N]: y
```

```
[16/16] Total
Importing OpenPGP key 0x621E9F35:
IDUsuario : "Docker Release (CE rpm) <docker@docker.com>"
Huella dactilar: 060A61C51B558A7F742B77AAC52FEB6B621E9F35
Desde : https://download.docker.com/linux/fedora/gpg
Is this ok [y/N]:
```

Una vez configurado se finalmente se instaló Docker en su última versión con el comando 'dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin'. Durante la ejecución del comando el repositorio solicitó la confirmación de una clave que había dentro del repositorio (con "y" o "N") la cual era 060A 61C5 1B55 8A7F 742B 77AA C52F EB6B 621E 9F35. Esta clave coincide con la que el manual requiere así que se respondió con "y".

```
tatiana@192:~$ sudo systemctl enable --now docker
[sudo] contraseña para tatiana:
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service -> /usr/lib/systemd/system/docker.service.
```



```
catiana@192:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbe9d7: Pull complete
Digest: sha256:a8dfb82aac212783bfc339d77d47ec32c8706ff250850eccc0e19c8737b18567
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Ya instalado se buscó inicializar Docker automáticamente sin tener que aplicar reinicio al servidor con el comando 'systemctl enable --now-docker' y crear un contenedor de prueba para verificar el correcto funcionamiento del servicio utilizando el comando 'docker run hello-world' siendo la acción exitosa.

10.1 Instalación de las imágenes de docker

Para nuestro proyecto hemos visto conveniente separar el funcionamiento de nuestra aplicación en dos partes: una parte para el área de la aplicación (área que incluye programación con lenguajes en JavaScript y PHP, diseño de vistas con front-controller, y bosquejos en front-end y back-end del sitio web) y la otra para la base de Datos (tablas creadas con sentencias de MySQL) ya que el funcionamiento de los contenedores de Docker no permiten más de una imagen en su sistema.

Las imágenes escogidas (mencionadas en la **bibliografía**) fueron extraídas de los repositorios oficiales de Docker Hub.

Para el contenedor de la aplicación se escogió la imagen php con el tag de la versión 8.2 y motor de Apache (php:8.2-apache) ya que nuestra aplicación está compuesta con esa versión del lenguaje y Apache es el "motor" y puerta de acceso para su ejecución y uso. Para el contenedor de la base de datos se escogió la imagen de MySQL con el tag "9-oraclelinux9" dado que eran de los pocos entornos que incluían un medio de funcionamiento con Linux y OracleLinux es de la línea RedHat.

Ambas imágenes fueron importadas al servidor con el comando 'docker pull'.

CEITAROX **I.S.B.O.** **3MJ**



```
tatiana@localhost:~$ sudo docker pull mysql:9-oraclelinux9
[sudol contraseña para tatiana:
Lo siento, pruebe otra vez.
[sudol contraseña para tatiana:
9-oraclelinux9: Pulling from library/mysql
500d7b2546c4: Pull complete
01859c60b4e2: Pull complete
87565b56b57e: Pull complete
9b2f3769f0be: Pull complete
7d70b564625b: Pull complete
1d289f7d1ed9: Pull complete
d210a5b69bfe: Pull complete
95f5cac1a9e9: Pull complete
98045e6cd572: Pull complete
1421f5b704d5: Pull complete
Digest: sha256:94254b456a6db9b56c83525a86bff4c7f1e52335f934cbcd686fe1ce763116a0
Status: Downloaded newer image for mysql:9-oraclelinux9
docker.io/library/mysql:9-oraclelinux9
```

```
tatiana@localhost:~$ sudo docker pull php:8.2-apache
8.2-apache: Pulling from library/php
ce1261c6d567: Pull complete
48ebd04f4983: Pull complete
70bb6b41f165: Pull complete
945024922144: Pull complete
2e432d6184b2: Pull complete
d4f032b51ed9: Pull complete
c4f07164d4bf: Pull complete
7ba7adab300a: Pull complete
08a8f3c4034a: Pull complete
aec45709115: Pull complete
80da6eb38538: Pull complete
5a7c38055062: Pull complete
9dcc75070870: Pull complete
1f8d884fef54: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:e66f0f7fcb0ff2c5fdccb662896521ac69312d3f5f18973ba3fac8e77b824c10
Status: Downloaded newer image for php:8.2-apache
docker.io/library/php:8.2-apache
```

11. Creación de los contenedores

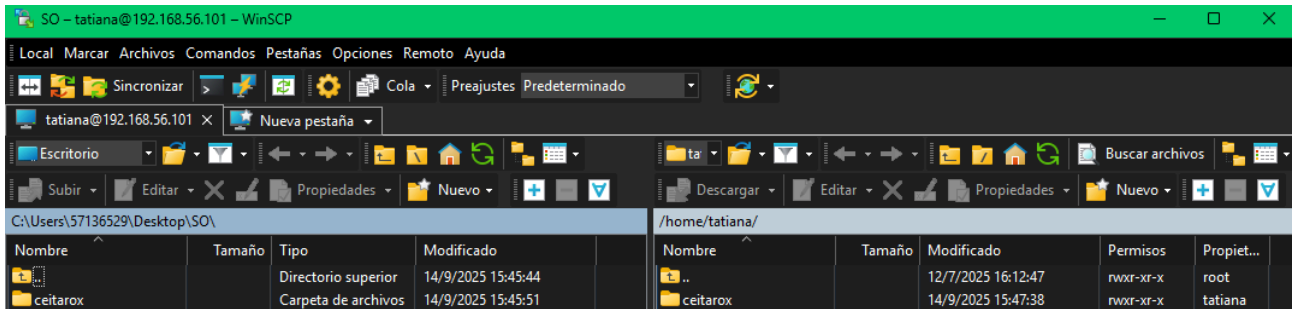
11.1 Contenedor con imagen de PHP 8.2 y Apache

```
tatiana@localhost:~$ sudo docker run -d --name aplicacion -p 80:80 php:8.2-apache
e0e5492736da633a3db7e6414aa40f701f514ba10a3591bd718670725a472b04
```

Durante el inicio en el delimitador -p queríamos utilizar el puerto 443 (HTTPS) como puerta de acceso y salida de la app web, pero debido a que esta imagen no admite tan fácilmente este protocolo (el servidor nos respondía desde las herramientas de desarrollador de Google Chrome, en el apartado de consola con el error 403) se escogió



al puerto 80 (HTTP). Para que el contenedor se ejecutara en primer plano se agregó también el delimitador -d.



Creado el contenedor se transfirió por protocolo SSH una carpeta llamada “ceitarox” que adjunta las carpetas utilizadas para el funcionamiento de la aplicación (carpeta “public” y “app”) al servidor.

```
tatiana@localhost:~$ sudo docker cp /home/tatiana/ceitarox/public aplicacion:/var/www/html
Successfully copied 43MB to aplicacion:/var/www/html
tatiana@localhost:~$ sudo docker cp /home/tatiana/ceitarox/app aplicacion:/var/www/html
Successfully copied 58.9kB to aplicacion:/var/www/html
```

Ya completada la transferencia de los archivos, con el comando ‘docker cp’ se copiaron las dos carpetas mencionadas al directorio de la carpeta pública del servicio Apache (interior del contenedor).

Uno de los problemas que tuvimos fue que el archivo index de la aplicación estaba almacenada en el directorio “public” y desde la configuración de Apache la carpeta pública solo está establecida únicamente en la carpeta “html”, pero no al resto de carpetas hijas. Nosotros probamos utilizando el delimitador -v en el comando ‘docker run’ para intentar definir esa “puerta de acceso” pero no nos dió buenos resultados .

Desde el artículo de PHP en Docker Hub encontramos que el archivo de configuración donde se establece esa limitación se encuentra en el directorio /etc/apache2/sites-available, en un archivo llamado “000-default.conf”.

```
tatiana@localhost:~$ sudo docker cp aplicacion:/etc/apache2/sites-available/000-default.conf /home/tatiana
[sudo] contraseña para tatiana:
Successfully copied 3.07kB to /home/tatiana
```

Dado que al ingresar en el contenedor con el comando ‘docker exec -it aplicación bash’ no permitía modificar este archivo (el archivo era de solo lectura y ni siquiera el contenedor permitía utilizar editores de texto) se tuvo que nuevamente utilizar el comando ‘docker cp’ para obtenerlo en el host, el cual es el almacenamiento de Fedora.

```
tatiana@localhost:~$ cp 000-default.conf 000-default1.conf
tatiana@localhost:~$ rm 000-default.conf
```

CEITAROX I.S.B.O. 3MJ



Hubo que copiar otra vez el archivo para quitarle los permisos “only-read” ya que no dejaba usar comandos de chmod y renombrarlo para luego eliminar el archivo con el nombre antiguo.

```
tatiana@localhost:~$ mv 000-default1.conf 000-default.conf
```

```
tatiana@localhost:~$ vim 000-default.conf
```

Ya duplicado se renombró al nombre que tenía antes y se ingresó al archivo para modificarlo.

```
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/public

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

Dentro del archivó se hizo una pequeña modificación en la propiedad ‘DocumentRoot’ para agregar la carpeta “public” a la dirección y así poder convertirla en una dirección accesible a la aplicación.

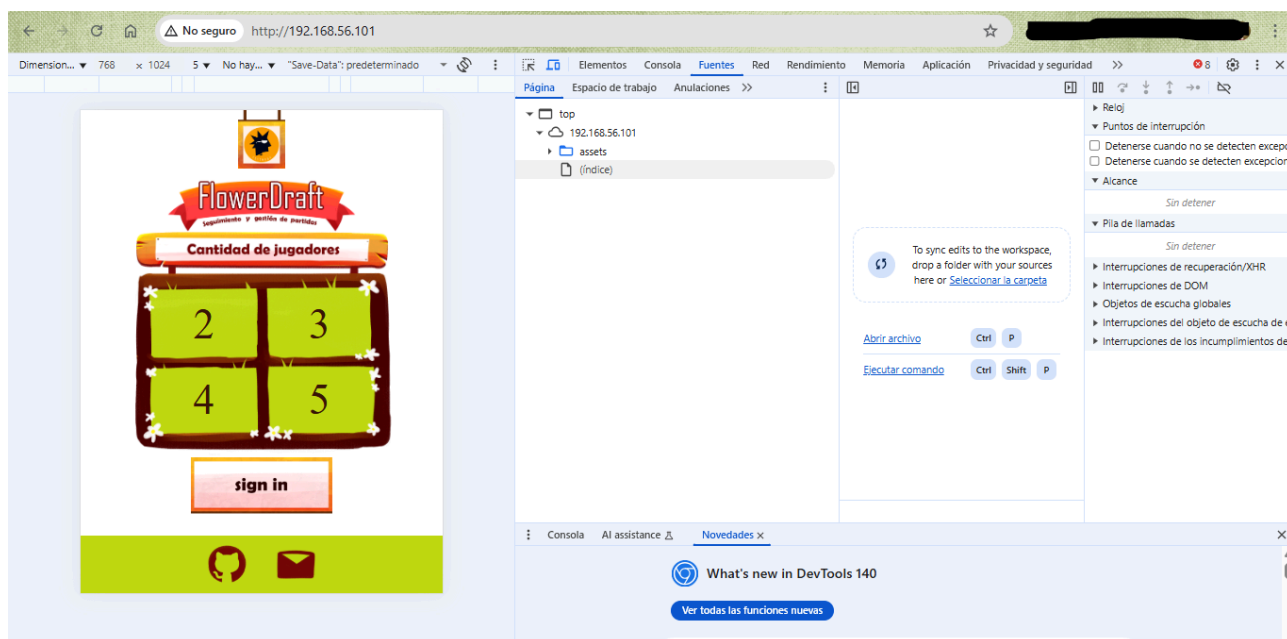
```
tatiana@localhost:~$ sudo docker cp /home/tatiana/000-default.conf aplicacion:/etc/apache2/sites-available
[sudo] contraseña para tatiana:
```

Una vez aplicado el cambio de configuración se copió nuevamente a su dirección original reemplazando el archivo de configuración antiguo.

```
tatiana@localhost:~$ sudo docker exec -it aplicacion service apache2 restart
Restarting Apache httsudo docker start aplicacion
```



Para aplicar completamente los cambios se reinicia el servicio Apache con el comando 'docker exec' con el delimitador -it para interactuar con el contenedor.



Gracias a ese archivo de configuración fue posible aplicar el primer gran adelanto a nuestro proyecto como la extensión de la accesibilidad remota.

11.2 Contenedor con imagen MySQL

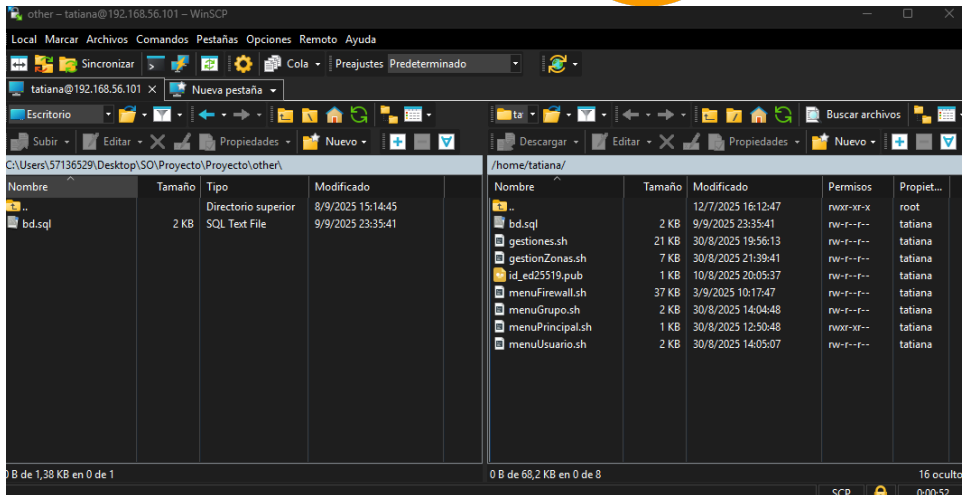
```
tatiana@localhost:~$ sudo docker run --name bd -e mysql_root_password=2t0@r8x -p 3306:3306 -d mysql:9-oraclelinux9-c9dc79a1bf26cb9f5f82b1fb058a891357b64176a6f29d619ba03deca1dfec75
```

Esta sentencia de 'docker run' es diferente a la anterior ya que requiere de otras especificaciones, por ser un contenedor que almacenará una base de dato se debe indicar el usuario encargado de la gestión (en este caso como pauta obligatoria de la imagen de MySQL había que definir la contraseña de acceso para root). El puerto de entrada y salida es el puerto de MySQL y la contraseña para el usuario root es "2t0@r8x".

CEITAROX I.S.B.O. 3MJ



15/09/25



Como se hizo con el contenedor con Apache primero se envió por SSH el archivo que contenía la base de datos al servidor para luego realizar otra copia para redirigirlo al contenedor de MySQL.

```
tatiana@localhost:~$ sudo docker cp /home/tatiana/bd.sql bd:/bd.sql
Successfully copied 3.07kB to bd:/bd.sql
```

Para importar el archivo .sql se ingresó al contenedor con 'docker exec' y se escribió esta sentencia:

```
bash-5.1# mysql -u root -p < /bd.sql
Enter password:
```

Lo que hace esta sentencia es loguearse a MySQL y añadir la base de datos bd.sql a las bases de datos del contenedor.

CEITAROX

I.S.B.O.

3MJ



```
mysql> show databases;
+-----+
| Database |
+-----+
| flowerdraftsystem |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.015 sec)

mysql> use flowerdraftsystem
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_flowerdraftsystem |
+-----+
| ficha |
| jugador |
| participa |
| partida |
| recinto |
| recinto_ficha |
+-----+
6 rows in set (0.016 sec)
```

Y como vemos este comando permite que se agreguen la base de datos y las tablas de la aplicación.

12. Enlace al repositorio de GitHub con los scripts y máquina virtual

13. Bibliografía

Docker. (n.d.). *Install Docker Docs*. Docker Inc. <https://docs.docker.com/engine/install/>

Docker Community. (n.d.). *php - Official Image Docker Hub: Changing DocumentRoot or other Apache configuration*. Docker Inc. https://hub.docker.com/_/php#changing-documentroot-or-other-apache-configuration

CEITAROX

I.S.B.O.

3MJ



15/09/25

Docker Community & MySQL Team. (n.d.). *mysql - Official Image Docker Hub*. Docker Inc. https://hub.docker.com/_/mysql

Axarnet. (n.d.). *Importar/exportar bases de datos MySQL*. Axarnet. <https://axarnet.es/blog/importar-exportar-bases-de-datos-mysql>

CEITAROX I.S.B.O. 3MJ